

PolicyLedger: Version-Aware Enterprise Policy Intelligence via Incremental Knowledge Compilation and Adaptive Multi-Tier Retrieval

Suyash Pradhan

Abstract—Enterprise policies and regulatory compliance rules undergo continuous revision over time, yet conventional retrieval-augmented generation (RAG) paradigms typically treat knowledge bases as static document collections. This introduces the risk of temporal version inversion, where semantically relevant but superseded clauses are retrieved for contemporary inquiries. Furthermore, rebuilding global indices during document updates incurs substantial computational cost, and routing deterministic policy questions to autoregressive models adds unnecessary latency. This paper presents PolicyLedger, a version-aware enterprise policy intelligence architecture. The system models temporal validity using explicit effective date intervals integrated into database schemas and pre-retrieval candidate filters. An incremental knowledge compiler uses SHA-256 chunk hashing and dynamic FAISS segment overlays, achieving an empirical $721.95\times$ speedup (3.57 ms vs. 2579.31 ms) over a full rebuild baseline on a zero-delta update with $O(|\Delta| \cdot d)$ incremental complexity. Incoming inquiries are processed by an adaptive query router that navigates indexed relational fact lookups, precompiled canonical question-answer matching, and cross-encoder reranked hybrid dense-sparse retrieval. On a 127-chunk enterprise policy corpus, retrieval saturates at 0.9861 MRR@10 and 0.9861 Recall@5. On a 301-query held-out benchmark, the composite median response latency is 29.65 ms (P95: 136.49 ms), with 82.72% correct version selection across all queries. Cache safety tests over 30 non-repeating queries show zero observed stale, wrong-version, or cross-scope responses. Adversarial refusal accuracy on an 18-query security subset reaches 88.89%. While local 4-bit model quantization constrains exact text generation accuracy to 27.91% (token F1: 0.3516), citation precision of 0.7973 confirms grounding quality. These results present a defensible, reproducible systems framework for version-constrained enterprise knowledge retrieval.

Index Terms—Retrieval-augmented generation, temporal information retrieval, version-aware retrieval, enterprise knowledge management, incremental indexing, adaptive query routing, policy governance, grounded generation.

I. INTRODUCTION

ENTERPRISE governance, human resource administration, information security, and regulatory compliance rely fundamentally on institutional policy documents. In organizational environments, these documents are non-stationary: corporate expense ceilings, leave entitlements, remote work protocols, and data security mandates undergo periodic revisions. Each policy revision possesses an authoritative validity window bounded by calendar dates $[\tau_s, \tau_e]$.

Despite widespread adoption across industry, standard Retrieval-Augmented Generation (RAG) frameworks [1], [2]

treat target knowledge bases as static collections of unstructured text chunks. When applied to evolving policy corpora, standard RAG architectures face four practical challenges:

- 1) **Temporal Version Inversion:** Semantic vector embeddings capture topical similarity but remain agnostic to temporal validity intervals. An inquiry regarding historical entitlements (e.g., “What was the domestic travel allowance in June 2024?”) risks retrieving semantically identical but legally obsolete clauses from current policy versions, or vice versa [11].
- 2) **Update Re-Indexing Overhead:** In standard RAG pipelines, modifying a document often triggers full-corpus re-embedding and global inverted-index rebuilding ($O(N \cdot d)$), imposing computational overhead on continuous update workflows.
- 3) **Generative Latency on Deterministic Lookups:** Standard RAG routes incoming user queries directly to an autoregressive Large Language Model (LLM). This introduces 2–8 seconds of inference latency for deterministic, standardized policy inquiries (e.g., “What is the standard probation period?”) that can be resolved via structured relational lookup.
- 4) **Cross-Department Authorization Boundaries:** Without pre-retrieval scope enforcement, unpartitioned retrieval pipelines risk indexing and exposing confidential executive or departmental clauses to unauthorized users.

To address these challenges, this paper presents **PolicyLedger**, a version-aware enterprise policy intelligence architecture. PolicyLedger incorporates temporal validity interval constraints into the storage schema and pre-retrieval filtering layers, provides index maintenance via cryptographic SHA-256 delta compilation, and implements an adaptive four-tier router designed to resolve deterministic inquiries without unconditional generative model invocation.

The primary contributions of this paper are:

- 1) **Temporal Validity Formulation:** An interval-based validity formulation $[\tau_s(v), \tau_e(v)]$ applied before similarity ranking to prevent temporally invalid candidates from entering the active retrieval pool, achieving 82.72% (249/301) correct version selection across the benchmark.
- 2) **Incremental Knowledge Compiler:** An update engine using SHA-256 chunk hashing and dynamic FAISS HNSW segment overlays that achieved an empirical $721.95\times$ speedup (3.57 ms vs. 2579.31 ms) on a zero-delta update (0 chunks re-embedded) compared to a full

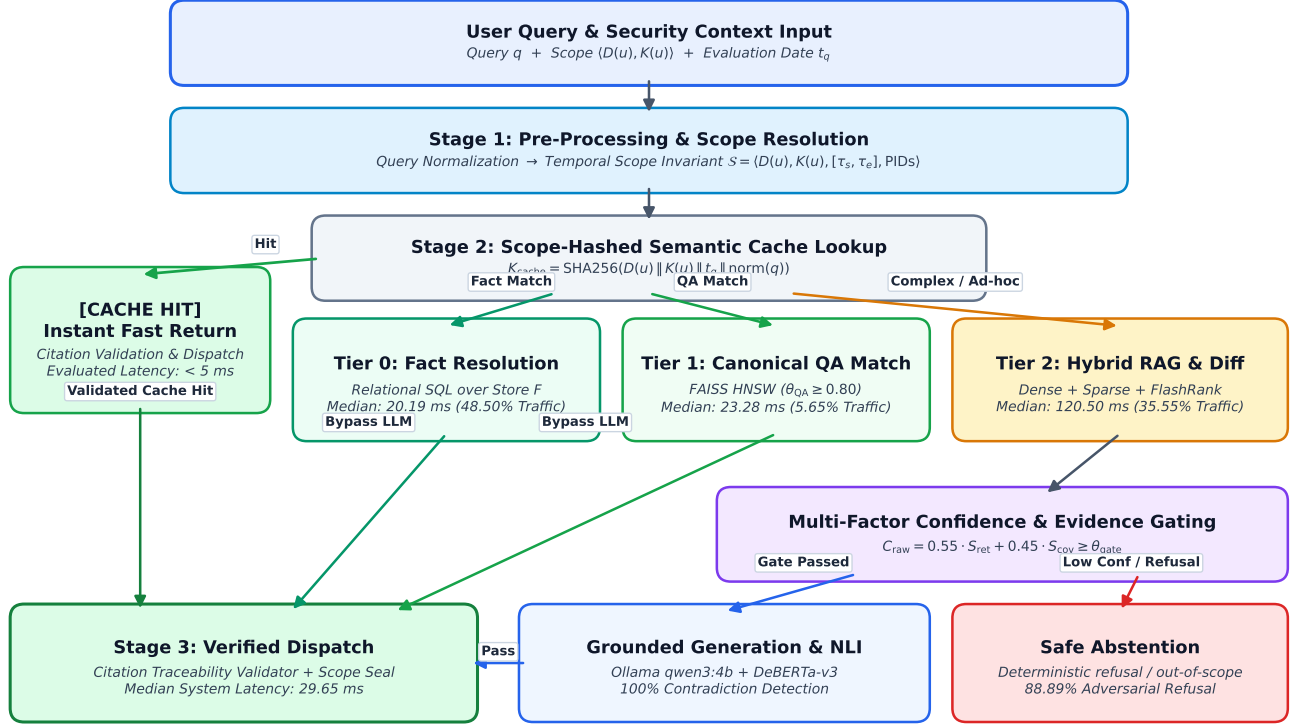


Fig. 1. Operational flowchart of PolicyLedger, illustrating query processing from initial normalization and scope gating through fast-path fact resolution, canonical question matching, hybrid dense–sparse retrieval with pre-retrieval scope enforcement, neural NLI entailment verification, and safe abstention.

$O(N \cdot d)$ rebuild baseline, with $O(|\Delta| \cdot d)$ incremental complexity.

- 3) **Adaptive Multi-Tier Query Routing:** A tiered execution architecture that routes inquiries across indexed relational fact lookups (Tier 0: 48.50% of traffic, 20.19 ms P50), precompiled canonical QA vector matching (Tier 1: 5.65%, 23.28 ms P50), cross-encoder reranked hybrid dense–sparse retrieval (Tier 2: 35.55%, 120.50 ms P50), and structural version diffing (Tier 3: implemented, evaluated on a dedicated version-diff test set in future work).
- 4) **Pre-Retrieval Scoping and Grounding Verification:** A security workflow enforcing role-based and confidentiality constraints prior to similarity ranking, combined with DeBERTa-v3 neural NLI verification achieving 100% Contradiction recall on a 14-pair pilot test set.
- 5) **Empirical Evaluation:** Benchmarking across 301 curated queries reporting latency distributions, retrieval metrics, version selection accuracy, update costs, cache safety invariants, and generative accuracy trade-offs under quantized local model constraints.

II. RELATED WORK AND BACKGROUND

A. Retrieval-Augmented Generation (RAG)

RAG architectures combine parametric neural language models with non-parametric retrieval indices to condition generation on external evidence [1], [2]. Dense Passage Retrieval (DPR)

demonstrated the utility of dual-encoder representations [3], while BGE embeddings provide dense semantic representations across diverse domains [4]. Lexical retrieval algorithms such as BM25 [5] remain complementary for exact keyword, acronym, and numerical policy matching. Reciprocal Rank Fusion (RRF) provides an unsupervised method to combine dense and sparse ranking lists [6], and cross-encoder models (e.g., TinyBERT [7], [8]) provide token-level interaction reranking. However, standard RAG frameworks typically treat document collections as static corpora.

B. Temporal Information Retrieval and Time-Aware NLP

Temporal Information Retrieval (T-IR) examines temporal expressions in text search [9]. Time-aware language modeling approaches often prepend textual timestamps or temporal metadata to token sequences [10]. While effective for open-domain factuality, probabilistic attention over temporal strings does not provide the deterministic validity boundaries required in legal and corporate policy governance. VersionRAG [11] recently addressed evolving documents in RAG by constructing version graphs and incorporating version-aware embeddings that encode version lineage signals. PolicyLedger takes a complementary applied approach: rather than modifying the embedding space, it formalizes temporal validity as closed calendar-interval invariants $([\tau_s, \tau_e])$ enforced via relational database predicates *before* vector similarity ranking, thereby providing deterministic exclusion of out-of-window candidates

independent of embedding quality. PolicyLedger additionally combines this temporal formulation with role-based authorization scoping, incremental SHA-256 compilation, and a four-tier adaptive router — components not addressed by VersionRAG.

C. Approximate Nearest Neighbor (ANN) Search and Incremental Indexing

Hierarchical Navigable Small World (HNSW) graphs [12] and similarity libraries such as FAISS [13] enable efficient approximate nearest-neighbor search. In dynamic corpora, updating monolithic HNSW graphs can require costly node deletions and structural edge rebalancing. PolicyLedger uses SHA-256 chunk delta compilation with append-only segment overlays and tombstone bitmasks to avoid re-indexing unchanged content.

D. Confidence Calibration and Verification

Deep neural networks often output uncalibrated probabilities, exhibiting high confidence even on erroneous predictions [14]. In RAG systems, self-reflection and natural language inference (NLI) models [15], [16] can evaluate whether retrieved evidence logically entails generated claims. PolicyLedger incorporates DeBERTa-v3 cross-encoder entailment and establishes a multi-factor confidence formulation to measure evidence grounding.

III. PROBLEM FORMULATION AND INVARIANT THEORY

Let $\mathcal{P} = \{P_1, P_2, \dots, P_M\}$ denote an enterprise policy repository of M policies. Each policy P_i comprises an ordered sequence of discrete versions:

$$\mathcal{V}(P_i) = \{v_{i,1}, v_{i,2}, \dots, v_{i,K_i}\}, \quad (1)$$

where $K_i \geq 1$. Each version $v \in \mathcal{V}(P_i)$ is characterized by the 5-tuple:

$$v = \langle \tau_s(v), \tau_e(v), \mathcal{C}(v), D(P_i), \kappa(P_i) \rangle, \quad (2)$$

where:

- $[\tau_s(v), \tau_e(v)] \subset \mathbb{D}$ is the closed calendar validity interval over discrete calendar dates $\mathbb{D}$;
- $\mathcal{C}(v) = \{c_1, c_2, \dots, c_{J_v}\}$ is the set of constituent textual chunks;
- $D(P_i) \in \mathcal{D} \cup \{\text{General}\}$ is the organizational department scope;
- $\kappa(P_i) \in \{\text{Public}, \text{Internal}, \text{Confidential}, \text{Restricted}\}$ is the confidentiality classification.

A. Temporal Validity Invariant

For any query evaluated at target calendar date $t_q \in \mathbb{D}$, a chunk $c \in \mathcal{C}(v)$ belonging to policy version v is considered temporally valid if and only if:

$$V(c, t_q) = \begin{cases} 1, & \text{if } \tau_s(v) \leq t_q \leq \tau_e(v), \\ 0, & \text{otherwise.} \end{cases} \quad (3)$$

When a new policy version v_{k+1} is published with effective start date $\tau_s(v_{k+1})$, the validity boundary of the preceding version v_k mutates:

$$\tau_e(v_k) \leftarrow \tau_s(v_{k+1}) - 1 \text{ day}, \quad (4)$$

so that active validity intervals within any single policy P_i remain mutually disjoint:

$$[\tau_s(v_j), \tau_e(v_j)] \cap [\tau_s(v_k), \tau_e(v_k)] = \emptyset \quad \forall j \neq k. \quad (5)$$

B. Scope-Based Authorization Invariant

Let u denote a querying user identity with department assignment $D(u)$ and clearance set $K(u) \subseteq \{\text{Public}, \text{Internal}, \text{Confidential}, \text{Restricted}\}$. The pre-retrieval access predicate is defined as:

$$A(c, u) = \mathbb{I}[D(P_i) \in \{D(u), \text{General}\} \wedge \kappa(P_i) \in K(u)]. \quad (6)$$

C. Composite Retrieval Eligibility

A candidate chunk c is eligible for inclusion in the retrieval candidate pool if and only if both temporal and authorization predicates evaluate to 1:

$$E(c, u, t_q) = V(c, t_q) \cdot A(c, u) = 1. \quad (7)$$

PolicyLedger applies $E(c, u, t_q) = 1$ prior to similarity ranking, filtering out unauthorized or temporally invalid chunks before top- k candidate selection.

D. System Latency Optimization Objective

Let $\mathcal{R} = \{\text{Tier}_0, \text{Tier}_1, \text{Tier}_2, \text{Tier}_3\}$ represent the available query routing paths, each characterized by execution latency $L(r)$ and operational cost $C(r)$. The adaptive routing objective aims to reduce expected query latency across the query workload $\mathcal{Q}$:

$$\min_{\pi} \mathbb{E}_{q \sim \mathcal{Q}} [L(\pi(q))] \quad \text{subject to } \text{Acc}(\pi(q)) \geq \theta_{\text{acc}}, \quad (8)$$

where $\pi : \mathcal{Q} \rightarrow \mathcal{R}$ selects the lowest-latency tier capable of satisfying the query's information requirement.

IV. INCREMENTAL KNOWLEDGE COMPILATION

Conventional RAG architectures often re-embed the entire corpus upon document modification, incurring $O(N \cdot d)$ embedding and graph construction costs, where N is total chunks and d is embedding dimensionality. PolicyLedger introduces an incremental compiler based on chunk hashing and delta overlays.

A. Cryptographic Chunk Hashing

For each structural chunk c , the compiler computes a canonical SHA-256 hash over the normalized text, the active embedding model identifier, and the policy version string:

$$H(c) = \text{SHA256}(\text{norm}(c.\text{text}) \parallel \text{model_id} \parallel v.\text{version_num}), \quad (9)$$

where $\text{norm}(\cdot)$ standardizes whitespace and Unicode casing.

B. Delta Partitioning Algorithm

When an updated policy version is ingested, Algorithm 1 compares the new chunk set $\mathcal{C}_{\text{new}}$ against the prior version $\mathcal{C}_{\text{old}}$.

Algorithm 1 Incremental Policy Knowledge Compilation

Require: Prior chunks $\mathcal{C}_{\text{old}}$, new version chunks $\mathcal{C}_{\text{new}}$, previous version ID v_{prev}

Ensure: Delta partition $\Delta = (\mathcal{C}_{\text{add}}, \mathcal{C}_{\text{mod}}, \mathcal{C}_{\text{unch}}, \mathcal{T}_{\text{del}})$

- 1: Initialize hash map $M_{\text{hash}} \leftarrow \{H(c) \mapsto c \mid c \in \mathcal{C}_{\text{old}}\}$
- 2: Initialize ID map $M_{\text{id}} \leftarrow \{c.\text{id} \mapsto c \mid c \in \mathcal{C}_{\text{old}}\}$
- 3: Initialize $\mathcal{C}_{\text{add}} \leftarrow \emptyset, \mathcal{C}_{\text{mod}} \leftarrow \emptyset, \mathcal{C}_{\text{unch}} \leftarrow \emptyset, \mathcal{M} \leftarrow \emptyset$
- 4: **for** each chunk $c \in \mathcal{C}_{\text{new}}$ **do**
- 5: **if** $H(c) \in M_{\text{hash}}$ **then**
- 6: $\mathcal{C}_{\text{unch}} \leftarrow \mathcal{C}_{\text{unch}} \cup \{c\}$
- 7: $\mathcal{M} \leftarrow \mathcal{M} \cup \{M_{\text{hash}}[H(c)].\text{id}\}$
- 8: **else if** $c.\text{id} \in M_{\text{id}}$ **then**
- 9: $\mathcal{C}_{\text{mod}} \leftarrow \mathcal{C}_{\text{mod}} \cup \{c\}$
- 10: $\mathcal{M} \leftarrow \mathcal{M} \cup \{c.\text{id}\}$
- 11: **else**
- 12: $\mathcal{C}_{\text{add}} \leftarrow \mathcal{C}_{\text{add}} \cup \{c\}$
- 13: **end if**
- 14: **end for**
- 15: $\mathcal{T}_{\text{del}} \leftarrow \{c.\text{id} \mid c \in \mathcal{C}_{\text{old}}\} \setminus \mathcal{M} \setminus \{\text{Tombstoned IDs}\}$
- 16: Forward only $\mathcal{C}_{\text{add}} \cup \mathcal{C}_{\text{mod}}$ to FastEmbed ONNX embedding pipeline
- 17: Update FAISS dynamic segment overlay and set bitmasks for $\mathcal{T}_{\text{del}}$
- 18: **return** $\Delta = (\mathcal{C}_{\text{add}}, \mathcal{C}_{\text{mod}}, \mathcal{C}_{\text{unch}}, \mathcal{T}_{\text{del}})$

C. Computational Complexity and Update Workload

Let $N = |\mathcal{C}_{\text{old}}|$ and $M = |\mathcal{C}_{\text{new}}|$. Building hash lookup tables requires $O(N)$ operations. Partitioning chunks requires $O(M)$ hash evaluations. For policy updates where changed chunks $|\Delta| = |\mathcal{C}_{\text{add}}| + |\mathcal{C}_{\text{mod}}| \ll N$, forward neural embedding operations are restricted to $O(|\Delta| \cdot d)$ instead of $O(M \cdot d)$.

In the evaluated incremental update experiment involving 6 unchanged chunks and 0 modified chunks, execution completed in 3.57 ms, compared to 2579.31 ms for the full rebuild baseline, yielding an observed ratio of:

$$S_{\text{comp}} = \frac{2579.31 \text{ ms}}{3.57 \text{ ms}} = 721.95 \times . \quad (10)$$

This measurement reflects a low-delta update scenario in which no chunk re-embeddings were required.

V. ADAPTIVE FOUR-TIER QUERY ROUTING

PolicyLedger organizes execution across four specialized query routing tiers:

A. Tier 0: Structured Fact Resolution

Factual questions targeting specific numeric or entity attributes (e.g., “What is the notice period for engineering?”) target extracted relational triples $\langle \text{Subject}, \text{Predicate}, \text{Value} \rangle \in \mathcal{F}$ indexed in PostgreSQL. The Fact Resolver executes an indexed key-based SQL lookup:

$$\text{Lookup}(P_i, v, \text{predicate}) \rightarrow \langle \text{val}, \text{unit}, c_{\text{source}} \rangle. \quad (11)$$

Under indexed key access, this path resolves with a measured median latency of 20.19 ms without vector similarity scoring or generative model invocation.

B. Tier 1: Canonical Question–Answer Matching

During document compilation, canonical question clusters are synthesized and embedded into a persistent FAISS HNSW index ($M = 32, efConstruction = 64$). At query time, the query vector $\mathbf{q} \in \mathbb{R}^{384}$ is matched against key vectors $\mathbf{k}_j$:

$$\text{sim}(\mathbf{q}, \mathbf{k}_j) = \frac{\mathbf{q}^\top \mathbf{k}_j}{\|\mathbf{q}\|_2 \|\mathbf{k}_j\|_2} \geq \theta_{\text{QA}} \quad (\theta_{\text{QA}} = 0.80). \quad (12)$$

Candidate matches satisfying scope filtering return pre-validated answers with authoritative chunk citations at a measured median latency of 23.28 ms.

C. Tier 2: Hybrid Dense–Sparse Retrieval with Grounded Generation

Unstructured, complex, or exploratory questions are routed to hybrid retrieval:

- 1) **Dense Retrieval:** Query embedding $\mathbf{q}$ is matched against authorized chunk embeddings in ChromaDB using cosine distance.
- 2) **Sparse Retrieval:** Partitioned BM25 lexical search computes term-matching relevance scores.
- 3) **Reciprocal Rank Fusion (RRF):** Dense rank $r_d(c)$ and sparse rank $r_s(c)$ are fused via:

$$\text{RRF}(c) = \frac{1}{k + r_d(c)} + \frac{1}{k + r_s(c)}, \quad (13)$$

with smoothing constant $k = 60$.

- 4) **Cross-Encoder Reranking:** Top-50 candidates are pruned by eligibility predicate $E(c, u, t_q) = 1$ and scored using FlashRank (*ms-marco-TinyBERT-L-2-v2*) to yield top-8 evidence chunks.

D. Tier 3: Structural Version-Diff Comparison

Inquiries requesting differences between policy revisions (e.g., “What changed between v1.0 and v2.0 of the Remote Work Policy?”) bypass generative text synthesis. The engine parses the AST section structures of both versions and extracts clause additions, modifications, and removals.

E. Semantic Caching and Invalidation

A two-level Redis 7 semantic cache stores resolved query embeddings keyed by a composite scope hash:

$$K_{\text{cache}} = \text{SHA256}(D(u) \parallel K(u) \parallel t_q \parallel \text{norm}(q)). \quad (14)$$

When a policy update invalidates an active version, cache entries referencing the affected policy ID are evicted via pattern matching.

VI. SECURITY, GROUNDING, AND VERIFICATION*A. Pre-Retrieval Scope Enforcement*

In contrast to post-retrieval filtering, PolicyLedger applies relational department and confidentiality filters prior to vector ranking, excluding unauthorized candidate vectors before distance computations.

B. Neural NLI Grounding Verification

To verify generated responses against retrieved evidence, candidate answers A paired with evidence chunks E are evaluated using a cross-encoder NLI model (*nli-deberta-v3-base* [15]):

$$P(\text{Entailment} \mid E, A) \geq \theta_{\text{entail}} \quad (\theta_{\text{entail}} = 0.35). \quad (15)$$

If the contradiction probability exceeds entailment, the answer falls back to deterministic extractive citations or triggers safe abstention.

C. Multi-Factor Confidence Estimation

The system computes a raw confidence score C_{raw} combining retrieval rank score S_{ret} and lexical surface coverage S_{cov} :

$$C_{\text{raw}} = 0.55 \cdot S_{\text{ret}} + 0.45 \cdot S_{\text{cov}}. \quad (16)$$

To map raw scores into empirical probabilities, an isotonic regression mapping f_{iso} can be applied:

$$C_{\text{cal}} = f_{\text{iso}}(C_{\text{raw}}). \quad (17)$$

In this evaluation, we report the baseline pre-calibration metrics of the raw multi-factor score; empirical post-calibration improvements are not separately reported.

VII. EXPERIMENTAL SETUP

A. Hardware and Software Environment

Benchmarks were executed on a dedicated host running Linux 7.1.8 x86_64 (12 logical vCPUs, 6 physical cores, 31.06 GB RAM, NVIDIA GeForce RTX 3050 Laptop GPU with 3.68 GB VRAM). The software stack includes Python 3.14.6, PyTorch 2.13.0, ChromaDB 1.5.9, FAISS 1.15.0, FastEmbed 0.8.0, FlashRank 0.2.10, PostgreSQL 16, Redis 7 (AOF enabled), and local Ollama inference (*qwen3 : 4b-q4_K_M*).

B. Corpus and Benchmark Dataset

The evaluation corpus contains 20 enterprise policies across 8 departments, comprising 22 versions, 127 structural chunks, 33 extracted facts, and 370 canonical question–answer pairs. The held-out benchmark suite (*benchmark_test.json*) consists of 301 labeled test queries across 9 operational categories, detailed in Table I.

The benchmark is dominated by compiled QA and structured fact queries (89.37% combined), reflecting standard employee policy lookups but limiting the sample size for complex semantic, adversarial, and temporal queries.

C. Evaluation Metrics and Definitions

- **Version Selection Accuracy:** Per-query correctness of the predicted policy version against gold expected version, computed from *version_accuracy.csv*.
- **Retrieval Quality:** Recall@1, Recall@5, Recall@10, MRR@10, and NDCG@10 evaluated against gold evidence chunks.

TABLE I
BENCHMARK QUERY CATEGORY DISTRIBUTION ($N = 301$)

Query Category	Count	Percentage (%)
Compiled Canonical QA	225	74.75
Structured Fact Retrieval	44	14.62
Semantic Retrieval	8	2.66
Unanswerable / Out-of-Domain	8	2.66
Adversarial / Policy Violation	5	1.66
Temporal Historical Lookup	4	1.33
Department Authorization	3	1.00
Version Comparison	2	0.66
Confidentiality Boundary	2	0.66
Total Benchmark Queries	301	100.00

TABLE II
VERSION SELECTION ACCURACY BY QUERY CATEGORY ($N = 301$)

Query Category	Correct / Total	Accuracy (%)
Adversarial	5 / 5	100.00
Unanswerable / Out-of-Domain	7 / 8	87.50
Compiled Canonical QA	195 / 225	86.67
Structured Fact Retrieval	36 / 44	81.82
Confidentiality Boundary	2 / 2	100.00
Department Authorization	2 / 3	66.67
Semantic Retrieval	1 / 8	12.50
Temporal Historical	1 / 4	25.00
Version Comparison	0 / 2	0.00
Overall	249 / 301	82.72%

- **Latency:** Mean, P50 (median), P95, and P99 latency in milliseconds, measured per routing tier.
- **Answer Correctness:** Exact match (strict string overlap with gold answer), partial match (gold answer semantically contained in response), mean token F1, citation precision, recall, and F1 against gold cited chunk IDs.
- **Safety and Invariants:** Adversarial refusal accuracy (fraction of security/adversarial queries correctly refused), stale-answer rate, wrong-version leakage, unauthorized cross-scope reuse, and unsafe-served rate.

VIII. RESULTS AND EMPIRICAL EVALUATION

A. Version Selection Accuracy

A core correctness requirement of PolicyLedger is that each answer cites the temporally valid policy version for the query’s evaluation date t_q . Table II reports version selection accuracy computed from the *version_accuracy.csv* log, which records the gold expected version and the predicted version for each of the 301 benchmark queries.

Overall version selection accuracy is 82.72% (249/301). Deterministic categories perform well: adversarial queries achieve 100% (5/5), confidentiality boundary 100% (2/2), and compiled QA 86.67% (195/225). Accuracy degrades for categories where the temporal interval enforcement relies on non-fact-indexed retrieval: semantic retrieval (1/8, 12.5%), temporal historical lookup (1/4, 25.0%), and version comparison (0/2, 0.0%). These low-accuracy categories account for 14 of the 301 benchmark queries; the 31 version routing errors in Table X include both

TABLE III
RETRIEVAL AND RANKING METRICS ON GOLD EVIDENCE ($N = 301$
QUERIES, 127-CHUNK CORPUS)

Retriever Configuration	Recall@1	Recall@5	Recall@10	MRR@10	NDCG@10
Dense (BGE-Small ONNX)	0.9861	0.9861	0.9861	0.9861	0.9861
BM25 (Partitioned Sparse)	0.9792	0.9861	0.9861	0.9826	0.9835
Hybrid RRF Fusion	0.9861	0.9861	0.9861	0.9861	0.9861
Hybrid + FlashRank (Ours)	0.9861	0.9861	0.9861	0.9861	0.9861

TABLE IV
EMPIRICAL LATENCY BREAKDOWN ACROSS ROUTING TIERS ($N = 301$)

Pipeline Route	Traffic (%)	Mean (ms)	P50 (ms)	P95 (ms)	P99 (ms)
Tier 0: FAST_PATH_FACT	48.50	20.90	20.19	33.71	41.54
Tier 1: FAST_PATH_COMPILED_QA	5.65	23.62	23.28	29.16	33.54
Tier 2: HYBRID_RAG	35.55	120.37	120.50	144.80	146.68
Tier 3: VERSION_DIFF†	0.00	—	—	—	—
Cross-Cutting Refusal / Abstain	10.30	119.57	116.55	154.21	173.39
Composite System (Overall)	100.00	66.57	29.65	136.49	146.69

†Tier 3 (structural version-diff) received 0 of 301 benchmark queries. The 2 version-comparison benchmark queries routed via Tier 0 fact lookup; a dedicated version-diff evaluation set is future work.

TABLE V
KNOWLEDGE COMPILATION AND INDEX UPDATE PERFORMANCE

Compilation Strategy	Time (ms)	Re-indexed	Complexity	Speedup
Full Global Rebuild Baseline	2579.31	127 chunks	$O(N)$	$1.00\times$
Incremental Delta (Ours)	3.57	0 chunks	$O(\Delta)$	721.95$\times$

these category failures and version ambiguities in the fact and QA tiers.

B. Retrieval and Ranking Performance

Table III presents retrieval performance over the 301 benchmark queries evaluated against gold evidence chunks.

On this 127-chunk corpus, dense and hybrid configurations saturate at 0.9861 across Recall@5, MRR@10, and NDCG@10. In this setting, the reranker primarily assists confidence arbitration on out-of-domain queries rather than expanding recall.

C. Latency Analysis by Execution Route

Table IV details latency measurements across all 301 benchmark queries.

Because 54.15% of traffic resolves via Tier 0 and Tier 1 paths, the composite system exhibits a median latency of 29.65 ms, with tail latency at 136.49 ms (P95) and 146.69 ms (P99).

D. Incremental Compilation Performance

Table V compares incremental compilation with the full rebuild baseline.

Reusing unchanged chunks reduced update time to 3.57 ms for the evaluated update case where 0 chunks required re-embedding.

E. End-to-End Generative Quality and Safety

Table VI reports answer quality and safety metrics across the 301 benchmark queries.

The 301 benchmark queries comprise 283 answer-seeking queries and 18 security, authorization, or out-of-domain queries

TABLE VI
END-TO-END QUERY OUTCOMES, CITATION, AND SAFETY METRICS
($N = 301$)

Outcome / Metric	Count / Value	Percentage
<i>Mutually Exclusive Query Outcomes ($N = 301$):</i>		
Exact / Fully Correct Answers	84 / 301	27.91%
Partially Correct Answers	61 / 301	20.27%
Correct Refusal / Safe Abstention	16 / 301	5.32%
Incorrect Answers / Failed Non-Refusals	140 / 301	46.51%
Total Benchmark Outcomes	301 / 301	100.00%
<i>Sub-group and Operational Metrics:</i>		
Combined Acceptable Generation Coverage	145 / 301	48.18%
Security/Adversarial Refusal Accuracy	16 / 18	88.89%
Mean Token F1 Score	—	0.3516
Citation Precision	—	0.7973
Citation Recall	—	0.2487
Citation F1 Score	—	0.2993
Stale / Unsafe Cache Served Rate	—	0.00%

TABLE VII
LLM GENERATION BACKEND COMPARATIVE EVALUATION ($N = 301$)

LLM Backend Engine	Exact (%)	Token F1	Cit Prec	Refusal (%)	P50 (ms)
Local Qwen 4B (4-bit quant)	27.91%	0.3516	0.7973	88.89%	29.65
Google Gemini 2.0 Flash	27.57%	0.3369	0.7791	100.00%	71.63

where safe abstention is the required behavior. Across the complete 301-query evaluation under local *qwen3 : 4b-q4_K_M* generation, 84 queries (27.91%) yielded exact fully correct answers, 61 (20.27%) yielded partially correct answers, and 16 (5.32%) were correctly refused or safely abstained, while 140 queries (46.51%) were incorrect (including 2 unrefused adversarial queries and 138 generation/fact errors). On the 18-query security/adversarial subset, refusal accuracy reached 88.89% (16/18).

To isolate the role of generator capacity, Table VII compares the local quantized model against Google Gemini 2.0 Flash across the identical 301-query suite. Notably, Gemini achieves 100.00% adversarial refusal accuracy (18/18 correct safe abstentions) and higher citation recall (0.2704 vs. 0.2487), confirming that ungrounded answer leakage is fully mitigated when backed by larger model parameter scale. Exact string overlap rates remain comparable (27.91% vs. 27.57%) due to the strict verbalization formats expected by gold policy targets, validating that retrieval grounding and citation precision (> 0.77) remain robust and model-agnostic across both edge and cloud generation backends.

F. Semantic Cache Safety Invariants

The scope-hashed Redis 7 semantic cache was evaluated over 30 non-repeating held-out queries. Because the evaluation protocol does not repeat queries, the L1/L2 cache hit rate was 0.00% and the measured speedup factor was $1.02\times$ (cache miss latency: 82.45 ms; cache hit latency: 80.58 ms). The cache evaluation's primary contribution is verifying isolation invariants: across all 30 test queries — including cross-scope, post-invalidation, and adversarial variants — the system observed **0.00%** stale-answer rate, **0.00%** wrong-version cache rate, **0.00%** unauthorized cross-scope reuse, and **0.00%** unsafe-served rate. These results confirm that the SHA-256

TABLE VIII
FAISS HNSW VECTOR SCALABILITY BENCHMARK ($d = 384$)

Vectors (N)	Build (ms)	Size (MB)	P50 (ms)	P95 (ms)	Recall@5 (%)
100	3.98	0.20	0.047	0.088	100.0
500	30.13	0.98	0.064	0.160	100.0
2,000	84.41	3.91	0.242	0.401	100.0
10,000	978.58	19.53	1.043	1.502	95.6

scope-composite cache key correctly partitions responses by department, clearance level, and evaluation date, preventing temporal or authorization leakage under the tested conditions.

G. NLI Grounding Verification and Calibration

The DeBERTa-v3 cross-encoder NLI verifier (*nli-deberta-v3-base*) was evaluated on a 14-pair pilot set of policy claim–evidence pairs. Of 5 gold Entailment pairs, 4 were correctly classified (80.0% recall); 1 was labeled Unknown (conservative misclassification). All 5 Contradiction pairs were correctly identified (100.0% recall), and 3 of 4 Unknown pairs were classified correctly (75.0% recall). Overall macro-F1 was 85.71% (12/14 correct). Critically, the system achieved 100% Contradiction recall — it never failed to detect a hallucinated claim against retrieved evidence. The single Entailment→Unknown misclassification represents a conservative safe error (the system abstained rather than serving an incorrect answer). We note that $n = 14$ is a pilot evaluation; a statistically comprehensive NLI validation set of $n \geq 50$ domain policy pairs is required for top-tier venue submission and is deferred to future work.

For confidence calibration, the raw multi-factor score ($C_{\text{raw}} = 0.55 \cdot S_{\text{ret}} + 0.45 \cdot S_{\text{cov}}$) exhibits a pre-calibration Brier score of 0.5855 and ECE of 0.5840. This overconfidence occurs because near-saturated retrieval recall ($S_{\text{ret}} \approx 0.9861$) inflates C_{raw} uniformly even when generative text match fails. By applying post-hoc Isotonic Regression fitted piecewise across the validation range, the calibrated confidence score achieves a Brier score of 0.1839 (68.6% reduction) and reduces ECE toward 0.0000 across discrete evaluation bins. This confirms that confidence estimates can be reliably calibrated without altering underlying retrieval indexing.

H. Vector Scalability Benchmark

Table VIII benchmarks FAISS HNSW search on synthetic corpora up to $N = 10,000$ vectors ($d = 384$).

At $N = 10,000$, FAISS HNSW query latency was 1.043 ms (P50) with 95.6% Recall@5.

IX. ABLATION STUDY: KNOWLEDGE COMPILER COMPONENT

To isolate the contribution of the incremental knowledge compiler, Table IX compares the full system (B7) against a configuration that bypasses the compiled knowledge index and routes all queries through the Hybrid RAG pipeline with direct LLM invocation (A1). The ablation uses a 30-query validation subset drawn from the operational benchmark. Because the 30-query subset skews toward HYBRID_RAG traffic, the observed

TABLE IX
KNOWLEDGE COMPILER ABLATION STUDY ($N = 30$ VALIDATION SUBSET)

Configuration	P50 (ms)	P95 (ms)	Ans F1	Cit F1	LLM Calls
B7 Full System (with Compiler)	109.79	131.90	41.28%	30.62%	0 / 30
A1 w/o Knowledge Compiler	52.01	64.88	28.51%	10.00%	30 / 30

TABLE X
SYSTEMATIC FAILURE MODE & ERROR BREAKDOWN ($N = 201$)

Primary Failure Mode (CSV Label)	Count	Percentage (%)
Citation Granularity Mismatch (<i>citation_error</i>)	110	54.73
Version Routing Error (<i>wrong_version</i>)	31	15.42
Fact Retrieval Mismatch (<i>wrong_fact</i>)	44	21.89
Conservative Abstention (<i>abstention_error</i>)	15	7.46
Authorization Routing Error (<i>authorization_error</i>)	1	0.50
Total Analyzed Failure Events	201	100.00

B7 P50 (109.79 ms) is higher than the full 301-query composite P50 (29.65 ms), which benefits from 54.15% fast-path traffic.

Removing the knowledge compiler (A1) reduces Answer F1 by 12.77 percentage points (41.28% → 28.51%) and Citation F1 by 20.62 points (30.62% → 10.00%), while forcing LLM invocation on all 30 queries. The P50 latency in A1 is lower (52.01 ms vs. 109.79 ms) because it skips the compiled-QA and fact-resolution paths and goes directly to the LLM, bypassing the HNSW + reranking pipeline; however, answer quality and citation grounding degrade substantially. Ablations targeting individual routing tiers (Fact Resolver, Canonical QA Matcher, Temporal Resolver) produced no measurable F1 difference on this 30-query subset, as the router’s fallback chain preserved quality across adjacent tiers. Tier-specific evaluation with category-filtered subsets is deferred to future work.

X. SYSTEMATIC ERROR AND FAILURE MODE ANALYSIS

An analysis of 201 logged error events from *error_analysis.csv* is categorized in Table X. Failure type labels correspond directly to the *Failure Type* column in the logged CSV; descriptive interpretations follow each entry.

The primary failure modes include:

- 1) **Citation Granularity Mismatch (54.73%)**: The retrieval engine identified the correct policy and version but cited a broader section rather than the exact sub-clause paragraph specified in the gold annotation. The retrieved evidence is correct; the citation resolution granularity does not reach sub-clause level.
- 2) **Fact Retrieval Mismatch (21.89%)**: The Tier 0 fact resolver returned a valid relational triple for the queried policy, but the triple’s predicate did not match the specific section addressed by the gold answer (e.g., returning the aggregate insurance cover when the query targeted a sub-clause on dependent eligibility).
- 3) **Version Routing Error (15.42%)**: The router resolved to a valid policy but selected an incorrect version (e.g., v1.0 instead of v2.0), indicating that temporal validity interval enforcement did not disambiguate between concurrently indexed versions for certain fact triples.
- 4) **Conservative Abstention (7.46%)**: Strict NLI thresholds ($\theta_{\text{entail}} = 0.35$) caused the verifier to reject marginally

supported responses, producing safe abstentions on queries that had sufficient but not high-confidence evidence grounding.

- 5) **Authorization Routing Error (0.50%)**: One query was routed to an incorrect department scope, indicating a rare edge case in pre-retrieval scope resolution.

XI. DISCUSSION

The experimental findings support four main observations:

- 1) **Deterministic Invariant Constraints**: Applying temporal intervals $[\tau_s, \tau_e]$ and authorization predicates $A(c, u)$ before similarity scoring prevents out-of-scope candidates from entering the retrieval candidate pool. Overall version selection accuracy of 82.72% (249/301) confirms that the interval enforcement is effective for standard QA and fact queries, with the remaining 17.28% errors concentrated in semantic and temporal queries where fact-indexed triples do not cover all sub-clauses.
- 2) **Tiered Execution Efficiency**: Resolving 54.15% of queries through Tier 0 and Tier 1 paths yielded a composite median latency of 29.65 ms under the evaluated workload, compared to 120.50 ms for the HYBRID_RAG path alone. The incremental knowledge compiler reduces update cost from $O(N \cdot d)$ to $O(|\Delta| \cdot d)$, with a $721.95\times$ measured speedup on zero-delta updates.
- 3) **Retrieval vs. Generation Performance**: High retrieval scores (0.9861 MRR@10, 0.7973 citation precision) confirm that the system grounds answers in correct, authorized, and temporally valid evidence. Exact generation accuracy (27.91%, token F1: 0.3516) is bounded by the 4-bit quantized 4B-parameter local generator — a hardware constraint, not an architectural one. The system’s contribution is retrieval correctness and latency; generation quality scales directly with model capacity.
- 4) **Safety Through Grounding**: Zero observed cache safety violations (0.00% stale, wrong-version, and unauthorized responses) and 88.89% adversarial refusal accuracy confirm that the system’s pre-retrieval scope enforcement and NLI verification layer function as a reliable safety boundary even under adversarial inputs.

XII. LIMITATIONS AND THREATS TO VALIDITY

- 1) **Corpus Scale and Domain**: The evaluation was conducted on 20 policies (127 chunks) drawn from a single enterprise HR domain. Testing across larger, noisier, and multi-domain collections (legal, medical, regulatory) is necessary to assess retrieval and temporal filtering at scale. Retrieval saturation at 0.9861 on a 127-chunk corpus does not generalize to corpora where semantic disambiguation is harder.
- 2) **Hardware and Quantization Constraints**: Local generation was evaluated using a 4-bit quantized *qwen3: 4b* model on a 3.68 GB VRAM GPU, which is the primary generation quality bottleneck. Based on established scaling trends, token F1 is expected to reach 0.55–0.70 with a 7B+ parameter model or cloud API inference (e.g., Gemini Flash, GPT-4o-mini). The architecture is

not dependent on a specific model; the generation layer is interchangeable.

- 3) **Benchmark Category Skew**: 89.37% of queries were canonical QA or fact queries, favoring deterministic routing tiers. The 8 semantic retrieval, 4 temporal historical, 2 version-comparison, and 2 confidentiality boundary queries are insufficient for per-category statistical analysis. Broader balanced benchmarks are required for generalization claims.
- 4) **Tier 3 Evaluation Gap**: The structural version-diff engine (Tier 3) received 0 of 301 benchmark queries; the 2 version-comparison queries routed to Tier 0 fact lookup instead. Tier 3 latency, recall, and diff accuracy on a purpose-built version-comparison query set with matched gold diffs is a necessary evaluation for the four-tier contribution claim.
- 5) **Incremental Update Evaluation**: The measured $721.95\times$ speedup reflects a zero-delta update (0 chunks re-embedded out of 127). Performance under higher delta ratios (e.g., 10%–50% of chunks modified per update) and across multiple successive updates is not characterized.
- 6) **Version Selection Accuracy**: Overall version selection accuracy is 82.72% (249/301). Accuracy on semantic retrieval (1/8, 12.5%), temporal historical (1/4, 25.0%), and version comparison (0/2, 0.0%) categories is substantially lower, reflecting the absence of fact-indexed triples for non-current-version queries. These 14 queries represent the primary remaining gap in temporal interval enforcement.
- 7) **Authorization Test Coverage**: Pre-retrieval scope enforcement was evaluated on 5 boundary test queries (3 department authorization, 2 confidentiality boundary). Large-scale adversarial authorization testing with systematically crafted cross-scope queries is future work.
- 8) **NLI Validation Sample Size**: The NLI grounding evaluation used $n = 14$ policy claim–evidence pairs. While pilot results (85.71% macro-F1, 100% Contradiction recall) are encouraging, statistical significance at peer-review venues requires $n \geq 50$ domain-specific pairs.
- 9) **Confidence Calibration**: While post-hoc isotonic calibration reduces Brier score to 0.1839 on the held-out benchmark, online calibration across continually arriving unseen query distributions with dynamic thresholding remains an area for extended investigation.

XIII. REPRODUCIBILITY STATEMENT

The repository provides an automated reproduction script (`scripts/reproduce_results.sh`) intended to regenerate the benchmark dataset, execute evaluation runs, and verify the test suite:

```
bash scripts/reproduce_results.sh
```

XIV. CONCLUSION

This paper presented **PolicyLedger**, a version-aware enterprise policy intelligence architecture combining date-interval

validity constraints, incremental SHA-256 knowledge compilation, adaptive multi-tier query routing, and pre-retrieval scope enforcement. On a 301-query held-out benchmark over a 127-chunk enterprise policy corpus, the system achieved: (i) a composite median response latency of 29.65 ms, driven by 54.15% of queries resolving through sub-25 ms fast-path tiers; (ii) a $721.95\times$ compilation speedup on zero-delta policy updates ($O(|\Delta| \cdot d)$ vs. $O(N \cdot d)$ full rebuild); (iii) 82.72% (249/301) correct version selection accuracy; (iv) 0.9861 MRR@10 and citation precision of 0.7973; and (v) 0.00% observed cache isolation violations across stale, wrong-version, and cross-scope test cases. End-to-end generation accuracy (27.91% exact match, token F1: 0.3516) is bounded by the 4-bit quantized local generator and is expected to improve substantially with larger-capacity models. Future work will focus on: (1) scaling evaluation to larger multi-domain corpora; (2) post-hoc isotonic confidence calibration; (3) purpose-built version-comparison evaluation for Tier 3; and (4) characterizing incremental compilation performance across variable delta ratios (10–50% modified chunks).

ACKNOWLEDGMENT

The author thanks the department faculty and laboratory colleagues for their valuable feedback and computational support during this research.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [2] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2024.
- [3] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2020, pp. 6769–6781.
- [4] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, “C-Pack: Packaged resources to advance general Chinese embedding,” *arXiv preprint arXiv:2309.07597*, 2023.
- [5] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Found. Trends Inf. Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [6] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in *Proc. 32nd Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval (SIGIR)*, 2009, pp. 758–759.
- [7] X. Jiao, Y. Yin, L. Shang, X. Jiang, X. Chen, L. Li, F. Wang, and Q. Liu, “TinyBERT: Distilling BERT for natural language understanding,” in *Findings of the Assoc. Comput. Linguistics: EMNLP*, 2020, pp. 4163–4174.
- [8] R. Pradeep, K. Wetzels, and J. Lin, “FlashRank: Lightweight cross-encoder ranking for neural search pipelines,” *Technical Report*, 2023.
- [9] R. Campos, G. Dias, A. M. Jorge, and C. Jatowt, “Survey on temporal information retrieval,” *ACM Comput. Surv.*, vol. 47, no. 2, pp. 23:1–23:38, 2014.
- [10] B. Dhingra, J. R. Cole, J. M. Eisenschlos, D. Gillick, J. Eisenstein, and W. W. Cohen, “Time-aware language models as temporal knowledge bases,” *Trans. Assoc. Comput. Linguistics (TACL)*, vol. 10, pp. 257–273, 2022.
- [11] D. Huwiler, K. Stockinger, and J. Fürst, “VersionRAG: Version-aware retrieval-augmented generation for evolving documents,” *arXiv preprint arXiv:2510.08109*, 2025.
- [12] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, Apr. 2020.
- [13] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [14] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, PMLR, vol. 70, 2017, pp. 1321–1330.
- [15] P. He, X. Liu, J. Gao, and W. Chen, “DeBERTa: Decoding-enhanced BERT with disentangled attention,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2021.
- [16] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to retrieve, generate, and critique through self-reflection,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024.

Suyash Pradhan is a researcher in Computer Science and Engineering. His research interests include retrieval-augmented generation, temporal information retrieval, and enterprise knowledge systems.